



NodeJS



01 | Introduction



Qasim Nadeem | Principal Technologist, ICT Innovative

Meet Your Instructor – Qasim Nadeem

- Microsoft Certified Professional in Many Web and Cloud Technologies
- Started Professional Life in 1989-90
- Spend 7 years in Low Level Software Engineering
- A Decade in Networks i.e. MCSE CCNA CCNP
- Last Decade in Web & Software Engineering i.e. HTML5, PHP, Java, Android, ASP.NET, C#, SQL Server, SharePoint, MongoDB ExpressJS Angular Node.js known as Mean Stack
- Running a Software House, Provides Consultancy to Ecommerce & Web Industry.
- Working in capacity of Principal Technologist & Head of Trainings

Who We Are?

Our team bears 30+ man years of experience in professional training and capacity building for 18,000+ individuals and 30+ organizations

Certified Trainers, State of the art labs/equipment, and Management-Owned high-class training facility makes us the prime choice

We focus on QUALITY and not QUANTITY

Special training programs for engineering graduates to impart 'Employable Skills' that lead to quick job placement.

Affiliations & Associations

Authorized Cisco
Network
Academy



Authorized
CERTIPORT®
Testing Center



Authorized
PEARSON VUE®
Testing Center

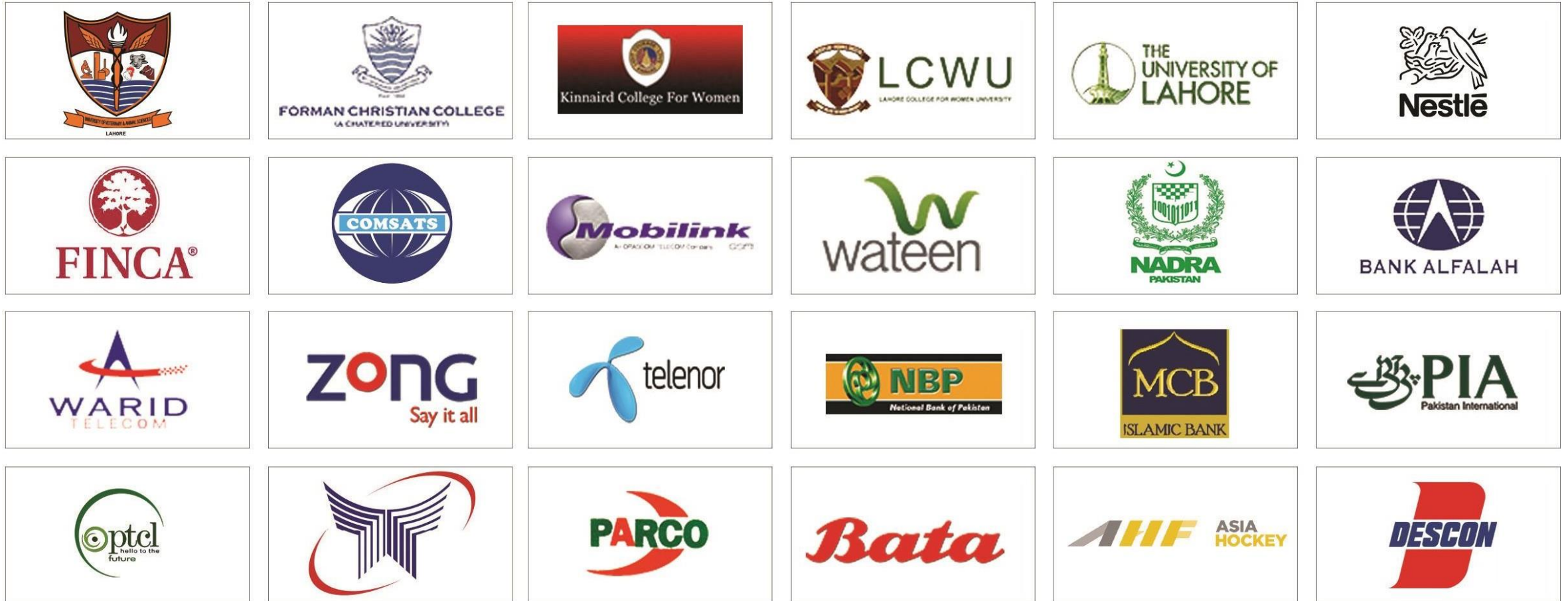


IELTS Partner
Program (IPP)
with British
Council



Selected Clients

(served directly or via business associates)



Why ICT Trainings?

- Credible: One of the training institution in Lahore to have 'training' and 'testing' partnerships simultaneously.
- Dependable: Trainers in ICT Trainings are all 'hands on' project leads, solution architects, and operations gurus. Learning from these personnel make the learning experience truly invaluable. They are all certified in their respective areas of expertise too.
- Quality: Our programs are pragmatic and practical. We ensure every trainees takes back home true learning experience that can actually facilitate in career enhancement and on-job promotions.
- Services Oriented: During the training programs, we provide dedicated and most updated lab infrastructure to trainees in order to truly augment their technical skills. Our trainers are always happy to coach trainees even after completion of training program.
- ON and OFF site Training: We are capable and flexible to deliver trainings to you in your location as well as at Lahore office. For ON site trainings, we are comfortable to move needed racks/equipment to your location too.
- FREE Vouchers: Having association as a learning partner with global technology inventors (e.g. Cisco Systems), we provide FREE exam vouchers to all our trainees against applicable training programs.

NodeJS



Course Topics

NodeJS

01 | Introduction to NodeJS

02 | Know the Environment

03 | NodeJS Console

04 | Welcome Application

05 | NodeJS as Web Server

06 | Handling Get Requests in Node

07 | Modules in NodeJS

08 | Express.js

09 | User-defined NPM Modules

10 | Routes

11 | Handling Post Request

12 | File Loading & Respond to Post Data

What is NodeJS?

Node.js is an open-source, cross-platform runtime environment used for development of server-side web applications. Node.js applications are written in JavaScript and can be run on a wide variety of operating systems.

Node.js is based on an event-driven architecture and a non-blocking Input/Output API that is designed to optimize an application's throughput and scalability for real-time web applications.

Over a long period of time, the framework available for web development were all based on a stateless model. A stateless model is where the data generated in one session (such as information about user settings and events that occurred) is not maintained for usage in the next session with that user.

A lot of work had to be done to maintain the session information between requests for a user. But with Node.js there is finally a way for web applications to have a real-time, two-way connections, where both the client and server can initiate communication, allowing them to exchange data freely.

Who use NodeJS?

Node.js is used by a variety of large companies, few are:

Paypal – A lot of sites within Paypal have also started the transition onto Node.js.

LinkedIn - LinkedIn is using Node.js to power their Mobile Servers, which powers the iPhone, Android, and Mobile Web products.

Mozilla has implemented Node.js to support browser APIs which has half a billion installs.

EBay hosts their HTTP API service in Node.js

Why use NodeJS? Features:

- Asynchronous event driven IO helps concurrent request handling – This is probably the biggest selling points of Node.js. This feature basically means that if a request is received by Node for some Input/Output operation, it will execute the operation in the background and continue with processing other requests.

This is quite different from other programming languages. A simple example of this is given in the code below

```
var fs = require('fs');  
    fs.readFile("Sample.txt",function(error,data)  
    {  
        console.log("Reading Data completed");  
    });
```

The above code snippet looks at reading a file called Sample.txt. In other programming languages, the next line of processing would only happen once the entire file is read.

But in the case of Node.js the important fraction of code to notice is the declaration of the function ('function(error,data)'). This is known as a callback function.

So what happens here is that the file reading operation will start in the background. And other processing can happen simultaneously while the file is being read. Once the file read operation is completed, this anonymous function will be called and the text "Reading Data completed" will be written to the console log.

Why use NodeJS?

- Node uses the V8 JavaScript Runtime engine, the one which is used by Google Chrome. Node has a wrapper over the JavaScript engine which makes the runtime engine much faster and hence processing of requests within Node also become faster.
- Handling of concurrent requests – Another key functionality of Node is the ability to handle concurrent connections with a very minimal overhead on a single process.
- The Node.js library used JavaScript – This is another important aspect of development in Node.js. A major part of the development community are already well versed in javascript, and hence, development in Node.js becomes easier for a developer who knows javascript.
- There are an Active and vibrant community for the Node.js framework. Because of the active community, there are always keys updates made available to the framework. This helps to keep the framework always up-to-date with the latest trends in web development.

When to use NodeJS

Node.js is best for usage in streaming or event-based real-time applications like:

Chat applications

Game servers – Fast and high-performance servers that need to process thousands of requests at a time, then this is an ideal framework.

Good for collaborative environment – This is good for environments which manage documents. In a document management environment you will have multiple people who post their documents and do constant changes by checking out and checking in documents. So Node.js is good for these environments because the event loop in Node.js can be triggered whenever documents are changed in a document managed environment.

Advertisement servers – Again here you could have thousands of requests to pull advertisements from the central server and Node.js can be an ideal framework to handle this.

Streaming servers – Another ideal scenario to use Node is for multimedia streaming servers wherein clients have requests to pull different multimedia contents from this server.

Node.js is good when you need high levels of concurrency but less amount of dedicated CPU time.

Best of all, since Node.js is built on JavaScript, it's best suited when you build client-side applications which are based on the same JavaScript framework.

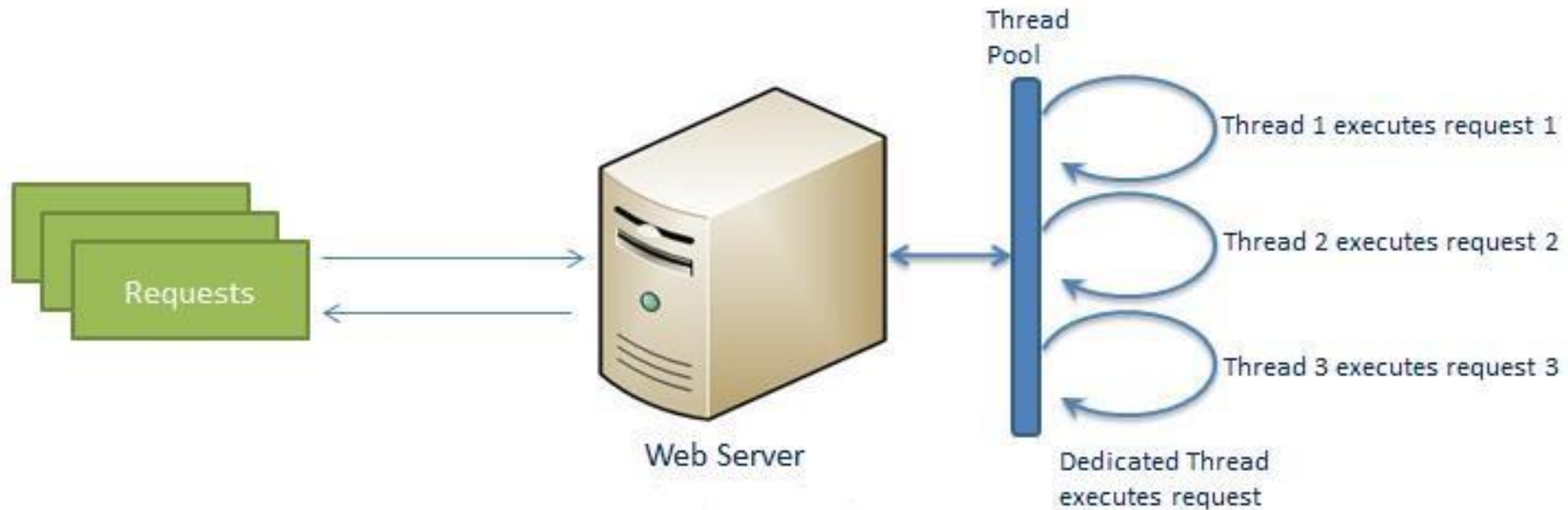
When not to use NodeJs

Node.js can be used for a lot of applications with various purpose, the only scenario where it should not be used is if there are long processing times which is required by the application.

Node is structured to be single threaded. If any application is required to carry out some long running calculations in the background. For example if the server is doing some calculation, it won't be able to process any other requests. Node.js is best when processing needs less dedicated CPU time.

Traditional vs NodeJS Process Model

Traditional web server model, each request is handled by a dedicated thread from the thread pool. If no thread is available in the thread pool at any point of time then the request waits till the next available thread. Dedicated thread executes a particular request and does not return to thread pool until it completes the execution and returns a response.



Traditional vs NodeJS Process Model

Node.js web server runs in a single process and the application code runs in a single thread and thereby needs less resources than other platforms.

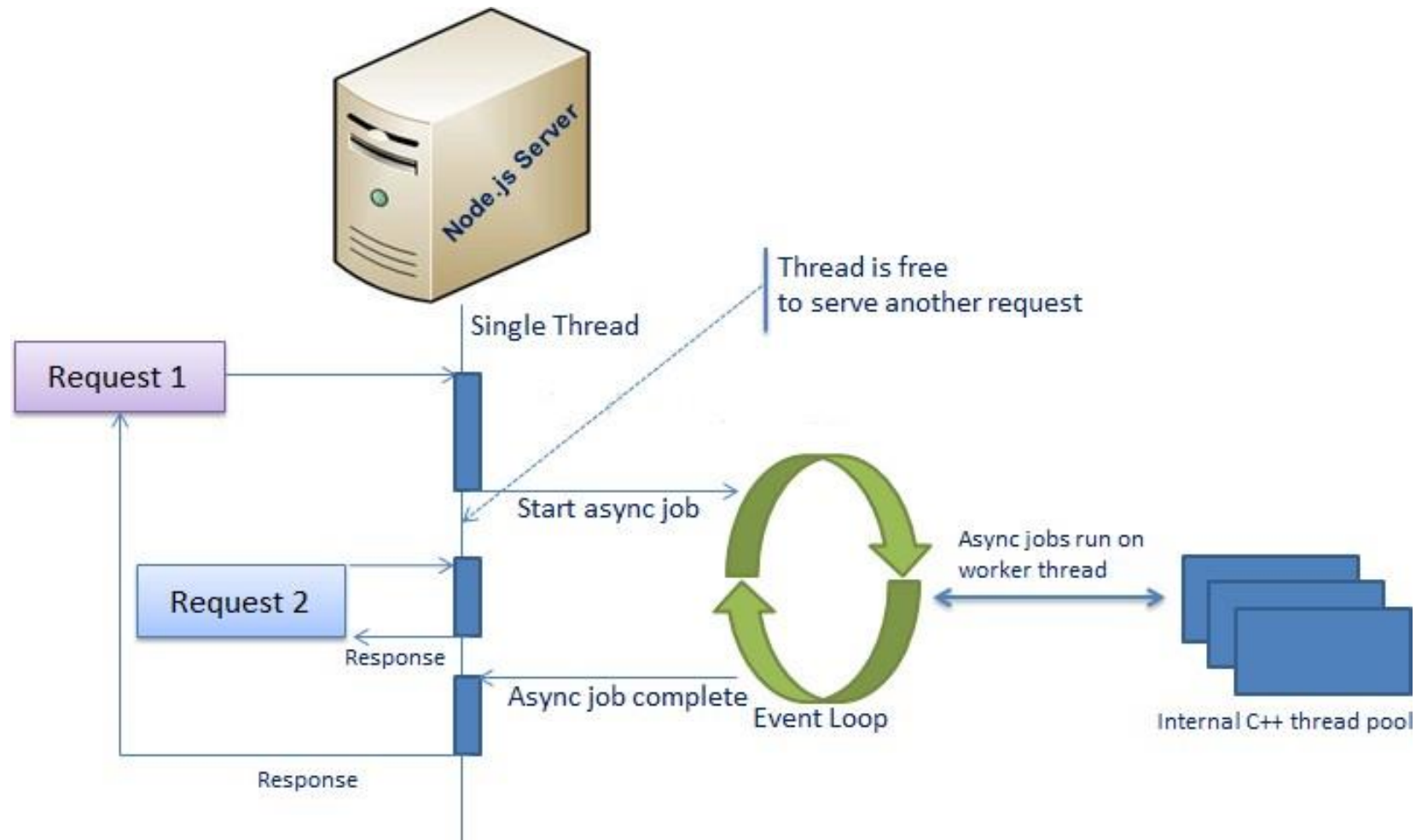
All the user requests to your web application will be handled by a single thread and all the I/O work or long running job is performed asynchronously for a particular request.

So, this single thread doesn't have to wait for the request to complete and is free to handle the next request. When asynchronous I/O work completes then it processes the request further and sends the response.

An event loop is constantly watching for the events to be raised for an asynchronous job and executing callback function when the job completes. Internally, Node.js uses libev for the event loop which in turn uses internal C++ thread pool to provide asynchronous I/O.

Concept illustrated on next slide:

Traditional vs NodeJS Process Model



Environment Setup

Node.js - <https://nodejs.org/en/download/>

//npm comes with node or install npm separately as follows:

Npm – This is known as the node package manager that is used to work with the open source repositories.

Steps:

- <http://npmjs.com> and go to the “get started with npm”.
- Choose the installer to download, depending on the operating system. Like download the Windows 64 bit version.
- Launch the installer -> click next -> Accept licence -> choose the destination folder for the installation -> click the Next button.
- Choose the components -> click next. You can accept all the components for the default installation.
- Once the installation is complete, click the Finish button.
- To confirm the installation, in the command prompt you can issue the command npm version. You will get the version number of npm.

NodeJS Console – Variable & Operator

You can now test pretty much any Node.js/JavaScript expression in REPL. For example, if you write "10 + 20" then it will display result 30 immediately in new line.

```
> 10 + 20
```

```
30
```

The + operator also concatenates strings as in browser's JavaScript.

```
> "Hello" + "World"
```

```
Hello World
```

You can also define variables and perform some operation on them.

```
> var x = 10, y = 20;
```

```
> x + y
```

```
30
```

NodeJS Console – Operator

Ternary:

```
var isCat = true;  
var pet = isCat ? "cat" : "dog";  
console.log(pet);
```

The code above generates the following result.

Bitwise:

Bitwise operations are supported in JavaScript: & (and), | (or), ~(inverse), and ^ (xor) operators.

```
> 20 | 10;
```

```
30
```

```
> 20 & 10;
```

```
0
```

```
> 10 & 10;
```

```
10
```

NodeJS Console – Data type

Node.js is highly performant, and it uses JavaScript because JavaScript supports first-class functions and closures.

Node.js has a few core types: number , boolean , string , and object.

The value undefined means that a value has not been set yet or simply does not exist:

```
var x;  
console.log(x);
```

Null
null is an explicit assertion that there "is no value":

```
var y;  
console.log(y);
```

```
y = null ;  
console.log(y);
```

typeof
To see the type of anything in JavaScript, use the typeof operator:

```
console.log(typeof 10);  
console.log(typeof "hello");  
console.log(typeof function () { var x = 20; });
```

NodeJS Console – Data type Number

All numbers in JavaScript are 64-bit IEEE 754 double-precision floating-point numbers.

All numbers in JavaScript have the same floating point number type.

Arithmetic operations (+,-,*,/,%) work on numbers as you would expect.

```
var myData = 1;  
var myValue = 2;
```

```
console.log(myData + 1);  
console.log(myData / myValue);  
console.log(myData * myValue);  
console.log(myData - myValue);  
console.log(myData % 2);
```

NodeJS Console – Data type Boolean

The boolean type Values in JavaScript can either be true or false.

Two literals are defined for boolean values: true and false.

We can convert values to boolean with the Boolean function, and the language converts everything to boolean when needed, according to the following rules:

false, 0 , empty strings "", NaN , null , and undefined all evaluate to false .

All other values evaluate to true .

```
console.log(0 == false);  
console.log("" == false);  
if(null){  
}else{  
    console.log("false");  
}
```

```
if(undefined){  
}else{  
    console.log("false");  
}  
if(NaN){  
}else{  
    console.log("false");  
}
```

NodeJS Console – Data type Strings

Strings in JavaScript are sequences of Unicode characters.

We use a string of length 1 to represent character.

Strings can be wrapped in single or double quotation marks. They are functionally equivalent.

To include a single quotation mark inside a single-quoted string, we can use `\'`, and similarly for double quotation marks inside double-quoted strings, we can use `\"`:

```
console.log('Javascript\'s new feature.')  
console.log("\"Hey, new feature!\", he said.")
```

```
var x = "sat";  
console.log(x.length);
```

NodeJS Console – Objects-1

To create an object, we can use either of the following,

```
var o1 = new Object();  
var o2 = {};
```

The latter, known as object literal syntax, is preferred.
We can specify the contents of objects using object literal syntax.
We can specify member names and values at initialization time:

Example

```
var user = {  
    first_name: "HTML",  
    last_name: "CSS",  
    age: 32,  
    website: "java2s.com"  
};
```

NodeJS Console – Objects-2

We can add a new property to your user object by using any of the following methods:

```
user.name= "brown";  
user["name"] = "brown";
```

```
var attribute = 'name';  
user[attribute] = "brown";
```

If we try to access a property that does not exist, we do not receive an error, but instead just get back undefined.

To remove a property from an object, we can use the delete keyword:

```
delete user.name;
```

NodeJS Console – Objects-3

The most common way of creating an object in JavaScript is using the object notation, {}.

Objects can be extended arbitrarily at runtime.

```
var myData = {};  
console.log(myData); // {}
```

```
myData.myValue = 123; // extend myData  
console.log(myData); // { myValue: 123 }
```

We can define which properties go on an object upfront by using the object literal notation.

```
var myData = {  
  myValue: 123  
};  
console.log(myData); // { myValue: 123 }
```

NodeJS Console – Objects-4

We can nest object literals inside object literals.

```
var myData = {  
  myValue: 123,  
  bas: {  
    bas1: 'some string',  
    bas2: 345  
  }  
};  
console.log(myData);
```

We can have arrays inside object literals.

```
var myData = {  
  myValue: 123,  
  bas: [1, 2, 3]  
};  
console.log(myData);
```

We can also have these arrays themselves contain object literals.

```
var myData = {  
  myValue: 123,  
  bas: [{  
    myItem: 1  
  },  
  {  
    myItem: 2  
  },  
  {  
    myItem: 3  
  }  
];  
console.log(myData.myValue); // 123  
console.log(myData.bas[0].myItem); // 1  
console.log(myData.bas[2].myItem); // 2
```

NodeJS Console – JSON

JSON, or JavaScript Object Notation is the data exchange format.

JSON is similar to object literal notation with two key differences.

In object literal notation, wrapping the property names in single or double quotation marks is optional, in JSON it is mandatory. All strings should be double-quoted in JSON as follows:

// valid object literal notation, INVALID JSON:

```
var obj = {  
  // JSON strings are supposed to use ", not '  
  "first_name": 'HTML',  
  // Must wrap property names for JSON  
  last_name: "CSS"  
}
```

// valid JSON and object literal notation:

```
var obj = {  
  "first_name": "HTML",  
  "last_name": "CSS"  
}
```

Note:

To generate JSON, we can use the V8 Javascript functions `JSON.parse` and `JSON.stringify`.

The former takes a JSON string and converts it to an object, while the latter takes an object and returns a JSON string representation of it.

NodeJS Console – Display Text

Create node-example.js file and add following code:

```
console.log("Hello World");
```

Execute node-example.js

```
>node node-example.js
```

Thus, you can execute any Node.js/JavaScript code in the node shell (REPL). This will give you a result which is similar to the one you will get in the console of Google Chrome browser.

NodeJS Console – Arrays

To create arrays, you can either use traditional notation or array literal syntax :

```
var arr1 = new Array();
```

```
var arr2 = [];
```

As with objects, the literal syntax version is preferred.

We can test if an object is an array using the `Array.isArray` function:

```
var arr2 = [];
```

```
Array.isArray(arr2);
```

```
Array.isArray({});
```

We can create arrays quite easily in JavaScript using `[]`.

Arrays have many useful functions.

```
var myData = [];
```

```
myData.push(1); // add at the end  
console.log(myData); // prints [1]
```

```
myData.unshift(2); // add to the top  
console.log(myData); // prints [2,1]
```

```
// Arrays are zero index based:  
console.log(myData[0]); // prints 2
```

NodeJS Console – Arrays

The array type in JavaScript length property returns the element count.

```
var arr2 = [];  
arr2.length;  
console.log(arr2);  
var arr3 = [ 'cat', 'rat', 'bat' ];  
console.log(arr3.length);
```

By default, arrays in JavaScript are numerically indexed:

```
var arr3 = [ 'cat', 'rat', 'bat' ];  
for (var i = 0; i < arr3.length; i++) {  
    console.log(arr3[i]);  
}
```

To add an item to the end of an array, you can do one of two things:

```
var arr3 = [ 'cat', 'rat', 'bat' ];  
arr3.push("mat");  
console.log(arr3);  
arr3[arr3.length] = "fat";  
console.log(arr3);
```

NodeJS Console – Arrays Insert-Delete

We can specify the index of the element where you want to insert a new element.

If this element is past the last element, the elements in between are created and initialized with the value undefined :

```
var arr3 = [ 'cat', 'rat', 'bat' ];  
arr3[20] = "splat";  
console.log(arr3);
```

To delete an item from an array, use the splice function, which takes an index and the number of items to delete.

What it returns is an array with the extracted items, and the original array is modified such that they no longer exist there:

```
var arr3 = [ 'cat', 'rat', 'bat', 'cat', 'rat', 'bat' ];  
arr3.splice(2, 2);  
console.log(arr3);  
console.log(arr3.length);
```

NodeJS Console – Array Functions

push and pop

The push and pop functions let you add and remove items to the end of an array, respectively:

```
var nums = [ 1, 1, 2, 3, 5, 8 ];  
nums.push(13);  
console.log(nums);  
nums.pop();  
console.log(nums);
```

unshift and shift

To insert or delete items from the front of an array, use unshift or shift , respectively:

```
var nums = [ 1, 2, 3, 5, 8 ];  
nums.unshift(1);  
console.log(nums);  
nums.shift();  
console.log(nums);
```

NodeJS Console – Array Functions

join

The array function join returns a string from the array:

```
var nums = [ 1, 1, 2, 3, 5, 8 ];  
var s = nums.join(", ");  
console.log(s);
```

The code above generates the following result.

Sort

You can sort arrays using the sort function, which can be used with the built-in sorting function:

```
var nums = [ 3, 1, 8, 5, 2, 1];  
nums.sort();  
console.log(nums);
```

We can provide your own sorting function as a parameter:

```
var names = [ 'CSS', 'HTML', 'Java', 'SQL', 'CSS3',  
              'HTML5'];  
names.sort();  
console.log(names);  
names.sort(function (a, b) {  
    var a1 = a.toLowerCase(), b1 = b.toLowerCase();  
    if (a1 < b1) return 1;  
    if (a1 > b1) return -1;  
    return 0;  
});  
console.log(names);
```

NodeJS Console - Functions

If you need to write multi line JavaScript expression or function then just press Enter whenever you want to write something in the next line as a continuation of your code. The REPL terminal will display three dots (...), it means you can continue on next line. Write .break to get out of continuity mode.

```
>function add(x,y)
... {
... return x+y;
... }
```

```
>add(10,40);
```

NodeJS Console – Loops

Create node-example.js file and add following code:

For Loop

```
>for(i=0; i<10; i++){  
... console.log("good");  
... }
```

For-in Loop

```
var user = {  
    first_name: "CSS",  
    last_name: "HTML",  
    age: 4,  
    website: "java2s.com"  
};
```

```
for (key in user) {  
    console.log(key);  
}
```

NodeJS Console – Loops

Create node-example.js file and add following code:

While Loop

```
> var i=1;  
  
> while(i<10){  
... console.log("goooooood");  
... i++;  
... }
```

Do-While Loop

```
> do{  
... console.log("good");  
... i++;  
... }while(i<10);
```

NodeJS Console – Loops

Create node-example.js file and add following code:

While Loop

```
> var i=1;  
  
> while(i<10){  
... console.log("goooooood");  
... i++;  
... }
```

Do-While Loop

```
> do{  
... console.log("good");  
... i++;  
... }while(i<10);
```

Welcome Application

Open the code in Visual Studio code.
Go to the folder → choose name → first.js. Add the following lines of code:

```
var http = require('http');  
http.createServer(function (req, res) {  
    res.writeHead(200, {'Content-Type': 'text/html'});  
    res.end('Hello World!');  
}).listen(8080);
```

Code Explanation:

The basic functionality of the "require" function is that it reads a JavaScript file, executes the file, and then proceeds to return an object. Using this object, one can then use the various functionalities available in the module called by the require function. So in our case, since we want to use the functionality of http and we are using the require(http) command.

In this 2nd line of code, we are creating a server application which is based on a simple function. This function is called, whenever a request is made to our server application.

When a request is received, we are asking our function to return a "Hello World" response to the client. The writeHead function is used to send header data to the client and while the end function will close the connection to the client.

We are then using the server.listen function to make our server application listen to client requests on port no 7000. You can specify any available port over here.

Node as WebServer using HTTP

Creating a simple server module which will listen on port no 7000. If a request is made through the browser on this port no, then server application will send a 'Hello' World' response to the client.

```
var http=require('http')
var
server=http.createServer((function(request,
response)
{
    response.writeHead(200,
        {"Content-Type" : "text/plain"});
    response.end("Hello World\n");
}));
server.listen(7000);
```

- The basic functionality of the require function is that it reads a JavaScript file, executes the file, and then proceeds to return the exports object. So in our case, since we want to use the functionality of the http module, we use the require function to get the required functions from the http module so that it can be used in our application.
- In this line of code, we are creating a server application which is based on a simple function. This function is called whenever a request is made to our server application.
- When a request is received, we are saying to send a response with a header type of '200.' This number is the normal response which is sent in an http header when a successful response is sent to the client.
- In the response itself, we are sending the string 'Hello World.'
- We are then using the server.listen function to make our server application listen to client requests on port no 7000. You can specify any available port over here.

Handling Get Requests in Node

Making a GET Request to get the data from another site is relatively very simple in Node.js. To make a Get request in the node, we need to first have the request module installed. This can be done by executing the command line

```
>npm install request
```

```
var request = require("request");
```

```
request("http://www.google.com",function(
    error,response,body)
    {
        console.log(body);
    });
```

- We are using the 'require' module which was installed in the last step. This module has the necessary functions which can be used to make GET requests to websites.
- We are making a GET Request [to www.google.com](http://www.google.com) and subsequently calling a function when a response is received. When a response is received the parameters(error, response, and body) will have the following values
 - Error – In case there is any error received when using the GET request, this will be recorded here.
 - Response- The response will have the http headers which are sent back in the response.
 - Body- The body will contain the entire content of the response sent by Google.
- In this, we are just writing the content received in the body parameter to the console.log file. So basically, whatever we get by going [to www.google.com](http://www.google.com) will be written to the console.log.

Modules in NodeJS

As stated earlier, modules in Node js are a way of encapsulating code in a separate logical unit. There are many readymade modules available in the market which can be used within Node js.

Below are some of the popular modules which are used in a Node js application

Express framework – Express is a minimal and flexible Node js web application framework that provides a robust set of features for the web and Mobile applications.

Socket.io - Socket.IO enables real-time bidirectional event-based communication. This module is good for creation of chatting based applications.

Pug - pug is a high-performance template engine and implemented with JavaScript for node and browsers.

MongoDB - The MongoDB Node.js driver is the officially supported node.js driver for MongoDB.

Restify - restify is a lightweight framework, similar to express for building REST APIs

Bluebird - Bluebird is a fully featured promise library with focus on innovative features and performance

What is Express?

Express.js is a Node.js web application server framework, which is specifically designed for building single-page, multi-page, and hybrid web applications.

It has become the standard server framework for node.js. Express is the backend part of something known as the MEAN stack.

The Express.js framework makes it very easy to develop an application which can be used to handle multiple types of requests like the GET, PUT, and POST and DELETE requests.

```
>npm install express //Install express
```

Following is the link to express examples: <https://expressjs.com/en/starter/examples.html>

Install Express

Assuming you've already installed Node.js, create a directory to hold your application, and make that your working directory is **myapp**

Use the npm init command to create a package.json file for your application.

```
$ npm init
```

This command prompts you for a number of things, such as the name and version of your application. For now, you can simply hit RETURN to accept the defaults for most of them, with the following exception:

Instead of entry point: (index.js) enter **app.js**, or whatever you want the name of the main file to be.

Now install Express in the myapp directory and save it in the dependencies list:

```
$ npm install express --save
```

Using Express in NodeJS

```
var express=require('express');
var app=express();

app.set('view engine','pug');
app.get('/',function(req,res)
{
    res.send('Hello World!')
});

app.listen(3000, () => {
    console.log(`Example app listening on
port 3000`)
})
```

- In the first statement itself, we are using the "require" keyword to include the express module. The "express" module is an optimized JavaScript library for Node.js development. This is one of the most commonly used Node.js modules.
- After the module is included, in order to use the functionality within the module, an object needs to be created. Here an object of the express module is created.
- Once the module is included using the "require" command and an "object" is created, the required methods of the express module can be invoked. Here we are using the set command to set the view engine, which is used to set the templating engine used in Node.js.

Express application generator

Express-generator is the application generator tool, it quickly create an application skeleton.

```
$ npx express-generator // available in Node.js 8.2.0
```

For earlier Node versions, install the application generator as a global npm package and then launch it:

```
$ npm install -g express-generator
```

```
$ express
```

On Windows Command Prompt, use this command: `set DEBUG=myapp:* & npm start` or just `npm start`

`localhost:3000/` in your browser to access the app.

Routes

Routing refers for determining the way in which an application responds to a client request to a particular endpoint.

route syntax: `app.METHOD(PATH, HANDLER)`

- `app` is an instance of `express`.
- `METHOD` is an HTTP request method: `get`, `post` etc.
- `PATH` is a path on the server.
- `HANDLER` is the function executed when the route is matched.

Example Route: `app.route('/Books').get(function(req,res){ res.send("Learning Routes"); });`

Now user make a http request for the above route with URL: `http://localhost:3000/Books`

So based on the URL which is accessed, a different functionality on the web server will be invoked and accordingly the response will be sent to the client.

Routes Example

```
var express=require('express');
var app=express();

app.route('/Node').get(function(req,res)
{
    res.send("Tutorial on Node");
});
app.route('/Angular').get(function(req,res)
{
    res.send("Tutorial on Angular");
});
app.route('/').get(function(req,res)
{
    res.send("Welcome to Trainings");
});
var server=app.listen(3000,function() {});
```

- Here we are defining a route if the URL `http://localhost:3000/Node` is selected in the browser. To the route, we are attaching a callback function which will be called when we browse to the Node URL.
- The function has 2 parameters.
- The main parameter we will be using is the 'res' parameter which can be used to send information back to the client.
- The 'req' parameter has information about the request being made. Sometimes additional parameters could be sent as part of the request being made, and hence the 'req' parameter can be used to find the additional parameters being sent.
- We are using the send function to send the string "Tutorial on Node" back to the client if the Node route is chosen.
- Here we are defining a route if the URL `http://localhost:3000/Angular` is selected in the browser. To the route, we are attaching a callback function which will be called when we browse to the Angular URL.
- We are using the send function to send the string "Tutorial on Angular" back to the client if the Angular route is chosen.
- This is the default route which is chosen when one browses to the route of the application – `http://localhost:3000`. When the default route is chosen, the message "Welcome to Trainings" will be sent to the client.

Handling Post Request – Body Parser

To handle HTTP POST request in Express.js, you need to install middleware module called body-parser. The middleware was a part of Express.js earlier but now you have to install it separately.

This body-parser module parses the JSON, buffer, string and url encoded data submitted using HTTP POST request. Install body-parser using NPM as shown below.});

```
npm install body-parser --save
```

Index File Loading & Respond to Post Data

```
var express = require('express'); // filename app.js
var app = express();
var bodyParser = require("body-parser");
app.use(bodyParser.urlencoded({ extended: false }));

app.get('/', function (req, res) {
  res.sendFile(__dirname+'index.html')
  ;
  //res.sendFile('index.html', { root:
  __dirname });
});
app.post('/submit-student-data', function (req, res) {
  var name = req.body.firstName + ' ' +
req.body.lastName;
  res.send(name + ' Submitted Successfully!');
});
var server = app.listen(5000, function () {
  console.log('Node server is running..');
});
```

```
<!DOCTYPE html> // index.html
<html>
<head>
  <meta charset="utf-8" />
  <title> </title>
</head>
<body>
  <form action="/submit-student-data"
method="post">
    First Name: <input name="firstName" type="text"
/>
    Last Name: <input name="lastName" type="text" />
    <input type="submit" />
  </form>
</body>
</html>

// npm install body-parser --save
```

Static Files in Express

To serve static files such as images, CSS files, and JavaScript files, use the `express.static` built-in middleware function in Express.

Syntax: `express.static(root, [options])` // root is directory from which to serve static assets.

For example, use the following code to serve images, CSS files, and JavaScript files in a directory named public: `app.use(express.static('public'))` //app is an express object

Note: To use multiple static assets directories, call the `express.static` middleware function multiple times

Virtual Path for static assest: Virtual path (means where the path does not actually exist in the file system) for files that are served by the `express.static` function: `app.use('/static', express.static('public'))`

Now, you can load the files that are in the public directory from the /static path prefix i.e.

`http://localhost:3000/static/images/kitten.jpg`

Absolute Path: If you run the express app from another directory, it's safer to use the absolute path of the directory that you want to serve:

```
const path = require('path')
app.use('/static', express.static(path.join(__dirname, 'public')))
```

Creating User Defined NPM Modules-1

Node.js has the ability to create custom modules and allows you to include those custom modules in your Node.js application.

Step 1) Create a file called "Addition.js" and include the below code. This file will contain the logic for your module.

```
var exports=module.exports={};  
exports.AddNumber=function(a,b)  
{  
  return a+b;  
};
```

- The "exports" keyword is used to ensure that the functionality defined in this file can actually be accessed by other files.
- We are then defining a function called 'AddNumber'. This function is defined to take 2 parameters, a and b. The function is added to the module "exports" to make the function as a public function that can be accessed by other application modules.
- We are finally making our function return the added value of the parameters.

Creating User Defined NPM Modules-2

Now that we have created our custom module which has the functionality of adding 2 numbers. It's now time to create an application, which will call this module.

Step 2) Create a file called "app.js," which is your main application file and add the below code

```
var Addition=require('./Addition.js');  
console.log(Addition.AddNumber(1,2));
```

We are using the "require" keyword to include the functionality in the Addition.js file.

Since the functions in the Addition.js file are now accessible, we can now make a call to the AddNumber function. In the function, we are passing 2 numbers as parameters. We are then displaying the value in the console.

```
>node app.js //to get an output
```

Extended User Defined NPM Modules-1

When creating modules, it is also possible to extend or inherit one module from another.

In modern day programming, it's quite common to build a library of common modules and then extend the functionality of these common modules if required.

Step 1) Create the base module.

```
var exports=module.exports={};  
exports.tutorial=function()  
{  
  console.log("Academy")  
}
```

The exports module is used so that whatever function is defined in this file can be available in other modules in Node.js. We are creating a function called tutorial which can be used in other Node.js modules. We are displaying a string "Academy" in the console when this function is called.

Extended User Defined NPM Modules-2

Step 2) Next we will create our extended module.
Create a new file called "NodeTutorial.js" and place the below code in the file.

```
var Tutor=require('./Tutorial.js');
exports.NodeTutorial=function()
{
  console.log("Node Tutorial")
  function pTutor()
  {
    var PTutor=Tutor
    PTutor.tutorial();
  }
}

Or
var Tutor=require('./Tutorial.js');
exports.NodeTutorial=function()
{
  console.log("Node Tutorial")
  this.pTutor = function ()
  {
    var PTutor=Tutor
    PTutor.tutorial();
  }
}
```

1.We are using the "require" function in the new module file itself. Since we are going to extend the existing module file "Tutorial.js", we need to first include it before extending it.

2.We then create a function called "Nodetutorial." This function will do 2 things,

3.It will send a string "Node Tutorial" to the console.

4.It will send the string "ICT Trainings" from the base module "Tutorial.js" to our extended module "NodeTutorial.js".

5.Here we are carrying out the first step to send a string to "Node Tutorial" to the console.

Extended User Defined NPM Modules-3

Step 3) Create your main app.js file which is your main application file and include the below code.

```
var localTutor=require('./NodeTutorial.js');  
localTutor.NodeTutorial();  
localTutor.NodeTutorial.pTutor();
```

Or use this code

```
var tut = new localTutor.NodeTutorial(); // Create and save object  
tut.pTutor(); // Call function on object
```

- Our main application file now calls the "NodeTutorial" module.
- We are calling the "NodeTutorial" function. By calling this function, the text "Node Tutorial" will be displayed in the console log.
- Since we have extended our Tutorial.js module and exposed a function called pTutor. It also calls the tutorial module in the Tutorial.js module, and the text "ICT Trainings" will be displayed to the console as well.

Publishing NPM Modules-1

One can publish their own module to their own Github repository.

By publishing your module to a central location, you are then not burdened with having to install yourself on every machine that requires it.

Instead, you can use the install command of npm and install your published npm module.

Step 1) Create your repository on GitHub (an online code repository management tool). It can be used for hosting your code repositories.

Step 2) You need to tell your local npm installation on who you are. Which means that we need to tell npm who is the author of this module, what is the email id and any company URL, which is available which needs to be associated with this id. All of these details will be added to your npm module when it is published.

Publishing NPM Modules-2

The below commands sets the name, email and URL of the author of the npm module.

```
npm set init.author.name "ICT."
```

```
npm set init.author.email "icttrainings@gmail.com"
```

```
npm set init.author.url http://ict-trainings.com
```

Step 3) The next step is to login into npm using the credentials provided in the last step. To login, you need to use the below command

```
npm login
```

Step 4) Initialize your package – The next step is to initialize the package to create the package.json file. This can be done by issuing the below command

```
npm init
```

Publishing NPM Modules-3

When you issue the above command, you will be prompted for some questions. The most important one is the version number for your module.

Step 5) Publish to GitHub – The next step is to publish your source files to GitHub. This can be done by running the below commands.

```
git add.  
git commit -m "Initial release"  
git tag v0.0.1  
git push origin master --tags
```

Step 6) Publish your module – The final bit is to publish your module into the npm registry. This is done via the below command.

```
npm publish
```

DEMO

Sample Applications

NodeJS Market



Local and International Market?

Pakistani Market:

<https://www.rozee.pk>

<https://www.jobee.pk>

International Market:

<https://www.careerone.com.au>

<https://www.bayt.com>

<https://www.upwork.com/>

And many more...

